

第07课：递归、DFS 与回溯

全排列 · 组合枚举 · 子集 · N 皇后 · 剪枝优化

一、学习目标

- 掌握 DFS 递归三要素：入口、出口、递归体
- 实现全排列、组合、子集枚举
- 理解回溯：选择 → 递归 → 恢复现场
- 应用剪枝减少搜索空间

二、核心知识点

递归：函数调用自身；需明确边界条件（出口）和递推关系；注意栈深度，n 过大可能溢出。

回溯：在决策树上 DFS；做选择前标记，递归返回后撤销标记（恢复现场），尝试其他分支。

全排列：n 个元素排列，用 used 数组标记已选，path 记录当前路径，pos 表示填第几位。

组合/子集：组合从 start 起选，避免重复；子集枚举可用二进制或 DFS 选/不选。

剪枝：提前判断分支不可行则 return，如 N 皇后攻击范围、和超过上限等。

三、常识与技巧

- 回溯模板：for 选择 → 标记 → dfs(下一层) → 撤销标记。
- 全排列输出格式注意题目要求（空格、换行、字典序）。
- N 皇后可用 attack 数组标记攻击位，或位运算优化。
- 记忆化 DFS：重复子问题存 dp，如 dfs-记忆化-P1962。

四、参考源码文件

本课内容整理自竞赛题库文件夹 2026fare，对应源码：递归-简单-P1706全排列问题.cpp、递归-简单-P10448组合型枚举.cpp、递归-简单-B3623排列型枚举.cpp、dfs-简单-B3622枚举子集.cpp、dfs-N皇后问题-P1219八皇后.cpp、dfs-组合-可行性剪枝-P1025数的划分.cpp、递归-素数-P1036选数1.cpp、递归-初阶-汉诺塔.cpp

五、代码示例与注释

示例 1：全排列 DFS (P1706)

```
vector<ll> path, used;
void dfs(ll pos) {
    if (pos > n) { // ■■■■■ n ■
        for (ll x : path) printf("%5lld", x);
        printf("\n");
        return;
    }
    for (int i = 1; i <= n; i++) {
```

